

Reinforcement Learning for Lunar Lander Trajectory Control

AE4350: Bio-Inspired Intelligence and Learning for
Aerospace Applications

Davide Roversi

Delft University of Technology

Reinforcement Learning for Lunar Lander Trajectory Control

by

Davide Roversi

Student Name	Student Number
D. Roversi	5737958

Public GitHub repository: https://github.com/davide-roversi/LunarLander_DQN.git
The repository contains code for nominal setup, sensitivity and robustness analyses,
saved agents, training data, plots, and landing animations.

Instructor: Dr.ir. E. van Kampen
Project Duration: June, 2023 - August, 2023
Faculty: Faculty of Aerospace Engineering, Delft

Cover: LM separation from CSM - Apollo Image Gallery
Style: TU Delft Report Style, with modifications by Daan Zwaneveld

Contents

Nomenclature	iii
1 Introduction and Problem Setup	1
1.1 Reinforcement Learning introduction	1
1.2 Lunar Lander environment	2
1.3 Code setup	3
2 Simulations and Results	3
2.1 Nominal case	4
2.2 Sensitivity Analysis	8
2.3 Robustness Analysis	9
3 Conclusions and Recommendations	9
References	11

List of Figures

1.1	Reinforcement learning loop ([12]). The agent interacts with the environment, collects rewards and updates its action policy.	1
1.2	2D-box environment. The black area represents flyable space, white is the ground, the red horizontal line indicates the landing pad.	2
2.1	Critic's neural network scheme.	6
2.2	Comparison of ReLU, leaky-ReLU and ELU activation functions. ([15])	6
2.3	Episode rewards and average rewards as function of training episodes for nominal problem setup. Training terminates at 204 episodes when the average reward reaches the value of 70.	7
2.4	Visualization of a successful landing simulation after the Agent has been correctly trained.	7
2.5	Average reward as function of episodes for different α values.	8
2.6	Average reward as function of episodes for different ϵ values.	8
2.7	Average reward as function of episodes for different ϵ_{decay} values.	9
2.8	Average reward as function of episodes for different Mini Batch Size values.	9
2.9	Average reward as function of episodes for different network layers sizes.	9
2.10	Average reward as function of episodes for different network update frequencies.	9
2.11	Robustness analysis results.	9

Nomenclature

Abbreviations

Abbreviation	Definition
RL	Reinforcement Learning
DQN	Deep Q-Network
MBS	Mini Batch Size
AC	Actor-Critic
PPO	Proximal Policy Optimization
GNC	Guidance, Navigation and Control

Symbols

Symbol	Definition	Unit
$\bar{s}_t$	State at time t	[m, m, m/s, m/s, -]
u_t	Action at time t	[N]
r_t	Reward at time t	[-]
$Q(s_t, u_t)$	Q-function (function of state and action)	[-]
α	Learning rate	[-]
γ	Discount factor	[-]
g	Gravitational acceleration	[m/s ²]

Introduction and Problem Setup

1.1. Reinforcement Learning introduction

Reinforcement Learning ([5]) is a branch of machine learning in which an Agent is trained through the experiences gathered from interactions with the environment. More specifically, based on the Agent's state and actions, rewards or penalties are collected. These are subsequently used to update the action choice, also called "policy", for the following timestep. The objective is to maximize the reward which, if the problem has been correctly set up, corresponds to the Agent having learned the task that was initially intended. The most general setup of an RL problem is shown in Fig. 1.1, where an "observation" is commonly referred to as a "state" $\bar{s}_t$ and actions are usually indicated with u_t . Depending on whether the state and the actions are described as continuous or discrete sets of values, different approaches can be used to solve the learning problem. As will be discussed in the following section, for the purpose of this project, the state space was chosen as continuous, while the actions are described as a finite set of discrete values. The chosen learning scheme was DQN (Deep Q-Network), commonly found in other implementations of the Lunar Lander problem ([16], [7], [4], [6], [3], [9]),

The underlying working principle behind DQN is that of Q-learning. This method is based on the definition of a Q-function $Q(\bar{s}, u)$, which is used to estimate the sum of future rewards. The Q-function depends on the current state $\bar{s}_t$ and chosen action u_t . When the Agent is trained correctly, it can pick the best action based on the predicted Q-function value that maximizes the reward. The Q-function expression is presented in Eq. 1.1, also known as the Bellman Equation ([6]).

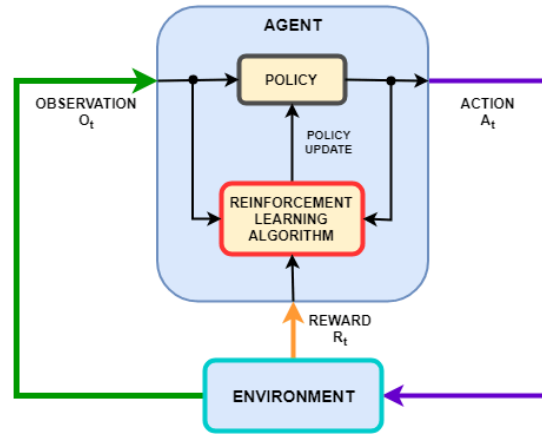


Figure 1.1: Reinforcement learning loop ([12]). The agent interacts with the environment, collects rewards and updates its action policy.

$$Q(\bar{s}_{t+\Delta t}, u_{t+\Delta t}) = Q(\bar{s}_t, u_t) + \alpha \cdot [r_t + \gamma \cdot \max Q(\bar{s}_{t+\Delta t}, u_t) - Q(\bar{s}_t, u_t)] \quad (1.1)$$

- $Q(\bar{s}_t, u_t)$: Q-function value at time t ;
- α : Learning rate. It dictates how significantly the value function gets updated across a step;
- $\max(Q(\bar{s}_{t+\Delta t}, u_t))$: Expected maximum value of the Q-function, given the current action, at the next state;

- γ : Discount factor. It scales the weight of the optimal Q-function value estimate on the update of the current Q-function value.

The quantity $(r_t + \gamma \cdot \max Q(\bar{s}_{t+\Delta t}, u_t) - Q(\bar{s}_t, u_t))$ is called "temporal difference" and directly represents the proportional change applied to the current Q-function value. It has a recursive form, being computed as the difference between the "target value" $r_t + \gamma \cdot \max Q(\bar{s}_{t+\Delta t}, u_t)$ and the Q-function value at the previous timestep $Q(\bar{s}_t, u_t)$. In theory, if the agent has been trained perfectly, the current Q-function value coincides with the sum of the current reward and the expected best future cumulative reward, bringing to zero the temporal difference value.

Based on these premises, DQN is built over standard Q-learning as a so-called *Off-Policy* approach that uses *indirect policy representation*. *Off-Policy* means that the Agent tries to improve a policy which can be different from the one it is using to make decisions ([12]). In other words, this involves learning from the behavior of an "older" policy, while simultaneously interacting with the environment using a newer, currently learned and improved policy ([10]). The main advantage of off-policy learning is that it allows for greater exploration of the environment, which can lead to better policies. *Indirect policy representation* means that the Agent is only composed by a critic and does not contain an actor. A critic returns the predicted discounted value of the cumulative long-term reward ([12]).

Deep Q-learning can be used both with discrete and continuous state representation, but requires a discrete action space. In case of a continuous state-space, a so-called function approximator is required. The most common, and the one used for this project, is a neural network. It must be noted that only one function approximator is required for a DQN problem since, as mentioned above, DQN employs a critic-only setup, with no separate actor.

In order for the Agent to actually learn, the algorithm must be set up with a balance of *exploration* and *exploitation*. *Exploration* refers to the Agent taking new actions that explore the action space, on the lookout for higher rewards. *Exploitation* is instead used to describe Agent actions that maximize rewards based on previous experience. The first is necessary to accumulate experiences, while the latter is crucial for the actual learning process. The chosen approach to balance these two aspects is the ϵ -greedy. At each timestep, the Agent has a probability ϵ (where $0 < \epsilon < 1$) of choosing a random action (exploration), and $1 - \epsilon$ of choosing an optimal-reward action based on a batch of previous experiences samples. The value of ϵ decays with time, orienting the Agent towards optimal-reward actions (greedy exploitation approach).

1.2. Lunar Lander environment

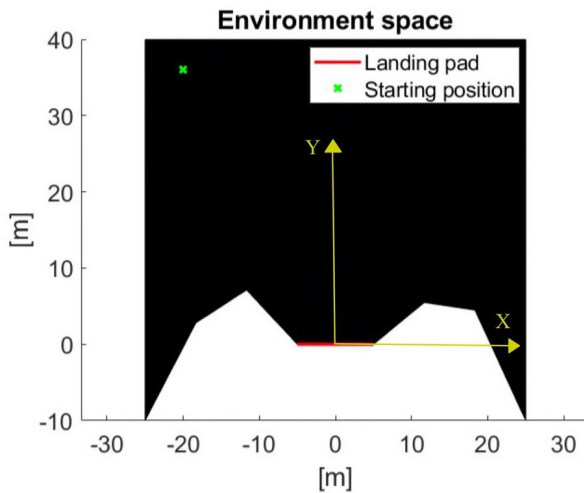


Figure 1.2: 2D-box environment. The black area represents flyable space, white is the ground, the red horizontal line indicates the landing pad.

The Lunar Lander environment has been defined as a 2D box-environment of size 50×50 m (Fig. 1.2). The landing pad is a strip of horizontal land of width 10 m, placed at the center of the coordinate frame (0,0). The x and y axes run horizontally and vertically respectively. The landscape around the landing pad is shaped as mountain peaks that can raise up to 15% of the total box height. All parameters for the environment dimensions are defined in the separate `config.m` file and can be easily adjusted by the user.

At the beginning of each simulation, the vehicle starts at the top-left corner of the environment and its objective is to land on the pad without crashing. In order to speed up the learning process, the vehicle fuel is assumed infinite and, therefore, the vehicle mass is fixed at a time-independent value. The environment exerts an acceleration of $g = 1.62 \text{ m/s}^2$ on the vehicle, constant in time and uniform in space in the negative- y direction. The vehicle is equipped with thrusters which can fire vertically (positive- y) to slow the descent, and laterally to adjust the hor-

horizontal position. Only translational motion is taken into consideration and the position of the vehicle is dictated by its center of mass. Based on these premises, the equations of motion that describe the vehicle position in time are those listed in Eq. 1.2, where T is the thrust, m the vehicle mass and g the lunar gravity acceleration.

$$\begin{cases} \frac{d^2x}{dt^2} = \frac{T_x}{m} \\ \frac{d^2y}{dt^2} = -g + \frac{T_y}{m} \end{cases} \quad (1.2)$$

The state $\bar{s}_t$ of the vehicle is a 5-element vector (Eq. 1.3). The first two elements (x, y) represent the spatial position in cartesian coordinates, the third and fourth (v_x, v_y) are the cartesian components for velocity, the final element c is used to store the flight condition of the vehicle. More specifically, $c = 0$ if the vehicle is in flight, $c = 1$ if it has impacted the ground (either within or outside the landing pad), and $c = 2$ if it has exited the flyable space boundaries.

$$\bar{s}_t = \begin{bmatrix} x \\ y \\ v_x \\ v_y \\ c \end{bmatrix} \rightarrow \frac{d\bar{s}}{dt} = \begin{bmatrix} v_x \\ v_y \\ T_x/m \\ -g + T_y/m \\ 0 \end{bmatrix} \quad (1.3)$$

The action space, being discrete, is a vector as well. It is composed by 5 elements, representing the finite set of actions that the Agent can take. Only one action can be taken at each timestep. Eq. 1.4 summarizes the action space, where $u_t = 0$ means that the vehicle does not take any action, $T_{x,low}$ and $T_{x,high}$ represent two different thrust levels for the main (vertical) engine, $\pm T_x$ indicates the direction and magnitude of the side thrusters.

$$\bar{u} = [0, T_{y,low}, T_{y,high}, +T_x, -T_x] \quad (1.4)$$

This choice of actions was made under the assumption that the side thrusters act as cold-gas thrusters would do in a real environment, with basically on-off control options. The main engine, on the other hand, was given three throttle levels (off-low-high) in order to provide the Agent with slightly finer vertical speed control.

1.3. Code setup

The code was entirely written in MATLAB, with the use of the Reinforcement Learning Toolbox ([13]) and Deep Learning Toolbox ([11]), which helped with the definition of the Agent parameters, learning setup and neural network definitions. The code is run from a `mainLunarLander.m` file which in turn makes use of multiple functions written in separate scripts.

The core structure of the main code is presented in Alg. 1, which contains three major blocks: the definition of environment, the setup of the network for the critic, and the creation of the agent based on the critic network with all the necessary parameters tuning. The Agent can then be trained and simulated. As introduced in previous sections, Agent training is an iterative and explorative process, where the agent adjust its policy based on the rewards and penalties collected during each run. Between runs, memory of previous experiences is kept to enable the learning process. Simulations, on the other hand, eliminate the exploration aspect and leave the agent to directly use the trained critic network for the action choices. As such, it is to be expected for the Agent behavior to be different between the last training round and the first actual simulation. However, if the training has been done correctly, the outcome should be closely similar, both in terms of trajectory and collected rewards.

For the setup of the environment and the subsequent learning runs, two functions are necessary: `funcReset.m` and `funcStep.m`. The former is used to initialize the simulation and sets the starting conditions, while the latter contains the computation of the state derivative, the forward integration step, all the necessary termination conditions checks, and the rewards allocations (Alg. 2).

All in all, for the definition of many parameters values, which will be presented in the following chapter together with the nominal results, an iterative approach had to be used. In most cases, values from similar problems and literature examples were chosen as a starting point, then tuned based on the knowledge of their impact on the algorithm, to obtain the desired outcome.

Algorithm 1 `mainLunarLander.m` code setup

- 1: Import configuration file;
 - 2: Define computing setup (GPU or CPU, parallelization);
 - 3: Define **Observations Space**, **Actions Space**, **Reset function**, **Step function**:
 - 4: → Create **environment**;
 - 5: → Define **critic neural network** parameters;
 - 6: → Create **Agent** from critic object and define its settings;
 - 7: Train the agent;
 - 8: Retrieve Training statistics and Trained Agent object;
 - 9: Simulate the trained Agent;
 - 10: **return** Training results, simulation results, plots.
-

Algorithm 2 `funcStep.m` code setup

- 1: Retrieve environment data;
 - 2: Retrieve state at current timestep;
 - 3: Retrieve chosen action at current timestep;
 - 4: Compute state derivative based on equations of motion and the state c value;
 - 5: Integrate the dynamics forward in time (Euler forward);
 - 6: Check on termination condition:
 - 7: **if** vehicle has exited the environment or touched the ground **then**
 - 8: Update final vehicle state;
 - 9: Save final velocity;
 - 10: Trigger termination condition to exit simulation;
 - 11: **end if**
 - 12: Compute rewards for current step;
 - 13: **return** current state, current step rewards, termination flag.
-

2

Simulations and Results

2.1. Nominal case

For the first part of the project, the objective was to find a nominal setup that would allow the vehicle to learn how to correctly land on the pad without crashing. A correct landing is any touchdown happening inside of the landing zone with a velocity norm (computed as $V = \sqrt{v_x^2 + v_y^2}$) lower than 2 m/s. Everything else is considered a crash. In this context, rewards and penalties are key parameters to ensure the correct functioning of the learning algorithm. Moreover, their definition is strictly related to the terminating events that trigger the start of a new simulation (also called "episode"). Each episode terminates either if the vehicle exits the flyable space or if it impacts on the ground. The main thought process behind rewards/penalties definition was to have a set of proportional values based on the vehicle state and action at each timestep, plus a set of constant values assigned upon termination of each episode. Tab. 2.1 summarizes the rewards and penalties laws used for this assignment. An episode reward $R_{episode}$ is also defined, at the end of each episode, as the sum of all rewards and penalties obtained during that run.

Table 2.1: Rewards and penalties setup for the Lunar Lander problem.

Definition	Evaluation	Reward/penalty function
Proportional to distance from target	Every timestep	$R = 0.001 \cdot ((\sqrt{x_{box}^2 + y_{box}^2}) - \sqrt{x(t)^2 + y(t)^2})$
Proportional to instantaneous velocity	Every timestep	$R = -0.01 \cdot (\sqrt{v_x(t)r + v_y(t)r})$
Proportional to side thrusters	Every timestep	$R = -0.03$ if $T_x(t) \neq 0$ $R = 0$ if $T_x(t) = 0$
Proportional to main engine	Every timestep	$R = -0.3$ if $T_y(t) \neq 0$ $R = 0$ if $T_y(t) = 0$
Exited boundaries	Upon termination	$R = -100$
Crashed outside of landing pad	Upon termination	$R = -50 - (\sqrt{v_x(t_f)^2 + v_y(t_f)^2})$
Crashed inside of landing pad	Upon termination	$R = -(\sqrt{v_x(t_f)^2 + v_y(t_f)^2})$
Landed correctly	Upon termination	$R = +100$

Table 2.2: Code parameters setup for the Lunar Lander problem.

Setup		Environment size [m]	[50, 50]
		Limit velocity [m/s]	2
		Action space [kN]	[0, 22, 32, 9, -9]
		Parallelization	no
		mainDevice	cpu
Agent Options	Epsilon Greedy Exploration	Use DDQN	Yes
		EpsilonDecay	0,05
		Epsilon	1
		EpsilonMin	0,001
	Critic Optimizer Options	LearnRate	1,00E-03
		GradientThreshold	1
		GradientThresholdMethod	l2norm
		L2Regularizationfactor	1,00E-04
		Algorithm	ADAM
		TargetSmoothfactor	1
		TargetUpdateFrequency	5
		ResetExperienceBufferBeforeTraining	0
		SequenceLength	1
		MiniBatchSize	128
		NumStepsToLookAhead	1
		ExperienceBufferLength	200000
		SampleTime	1
		DiscountFactor	0,99
Network		Number of Layers	2
		Neurons per layer	[256, 256]
		Activation functions	[leakyRelu, relu]
		WeightInitializer	[glorot, glorot]
		BiasInitializer	[zeros, zeros]
Termination criteria		StopTrainingCriteria	AverageReward
		ScoreAveragingWindowLength	50
		StopTrainingValue	70

On top of the rewards definition, all the code parameters had to be defined for the Agent, the critic's neural network, and the termination conditions. Tab. 2.2 gathers all the code settings that were needed to effectively allow the code to run. As can be seen, the main engine thrust levels were defined much higher compared to the side thrusters, even on low throttle setting. This was done because the main engine acts against gravity to control the descent, while the side thrusters have no opposing force. Also, the "DDQN" setting is turned on, meaning that the training actually makes use of the Double Deep Q-Learning variant of DQN. This approach improves convergence stability by reducing a common Q-function value overestimation issue that characterizes most DQN applications. This is done by modifying the definition of the target equation to $r_t + \gamma \cdot Q \cdot (s_{t+\Delta t} \cdot \operatorname{argmax}(Q(s_{t+\Delta t}, u_t)))$ ([3]).

As for the other settings, ϵ_{decay} is the parameter dictating how much the greedy action choice prevails over the random exploration, following the linear law $\epsilon_{i+1} = \epsilon_i \cdot (1 - \epsilon_{decay})$ ([14]). Setting it to $\epsilon_{decay} = 0.05$ allows to reach ϵ_{min} in around 135 iterations, which was deemed a viable amount after trial and error runs. As for the optimizer algorithm used to update the critic's neural network, acceptable results were obtained by keeping the default settings. More specifically, the ADAM optimizer is used. It is an adaptive version of the stochastic gradient descent method, which adjusts the learning rate α as the simulations progress ([8]). As for the target update frequency, examples found on the web mostly kept its value between 1 and 5 ([7], [9], [1]), with only one going up to 20 ([16]). For this assignment, 5 proved to work the best. The Mini Batch Size value represents the size of the experiences pool (stored in the replay memory buffer) from which the agent samples to compute gradients for critic updates ([14]). In most examples of the Lunar Lander problem, 64 ([16],[7]) or 128 ([4], [6]) seem to be preferred values. For this assignment, 128 appeared to provide the best performance. The discount factor was kept at the default $\gamma = 0.99$, which is also the most popular setting found in other examples and literature ([7], [6], [3], [9], [1]). This is a considerably high value, which means that even the estimate of far future rewards is taken into high consideration, when deciding how to update the Q-function value.

The network settings are another crucial aspect that needs to be correctly tuned. Too simple of a network might not be able to correctly capture the system dynamics, while an excessively complex one may lead to unfeasible computational time and overfitting. In case of the latter, the risk is that the solutions found loose generality and do not work if applied to slightly different cases. One of the main choices lies in the allocation of neurons per-layer and the number of layers. Generally, a deeper network (i.e. more layers) has a higher level of abstraction, meaning it is able to capture more correlations in the input data ([2]). Given the overall simplicity of the problem's dynamics, 2 layers with 256 neurons each (Fig. 2.1) proved sufficient. Nonetheless, examples of similar problems show a variety of network settings being used ([16], [4], [3], [9]), suggesting that this is one field in which there is considerable room for tuning and flexibility. Conversely, more agreement appears in the choice of activation functions, where the ReLU is most common ([4], [6], [9], [1]). However, for this project, it was discovered that a leaky-ReLU (Fig. 2.2) on the first layer significantly improved performance, likely thanks to its ability in "filtering out" less of the input information.

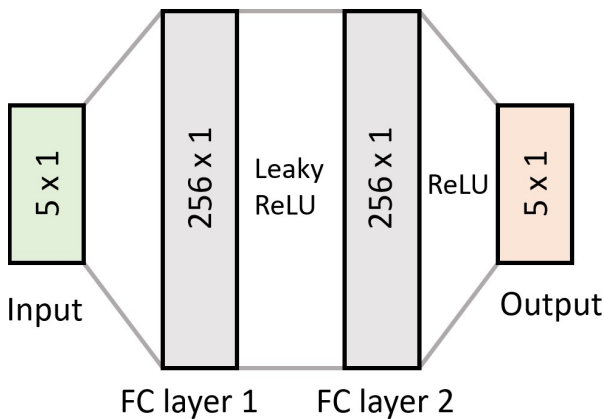


Figure 2.1: Critic's neural network scheme.

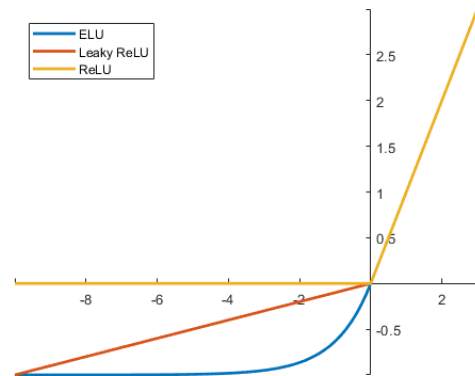


Figure 2.2: Comparison of ReLU, leaky-ReLU and ELU activation functions. ([15])

Finally, for the termination conditions, a rolling episode reward average of $R_{episode} > 70$, computed over 50 consecutive runs, was deemed sufficient to assess the convergence of the learning algorithm. A maximum of 1500 runs was also set to prevent non-converging simulations for running too long.

Training the agent with the above mentioned settings shows convergence in 204 runs (Fig. 2.3). The episode rewards are consistently negative in the first 50 runs, then the Agent starts learning and the number of positives starts growing. The average rewards trend is overall increasing, indicating a smooth progress. Fig. 2.4 shows after-training simulation results in terms of environment parameters. As can be seen, at the end of the training, the Agent has learned to fire its side thrusters to start aligning with the landing pad, initially letting itself fall under the effect of gravity. About mid way through the trajectory, the main engine is activated, both on the low and on the high throttle settings, to slow the descent below the maximum allowed threshold. Simultaneously, a side thruster is fired intermittently to bring the vehicle closer to the center of the landing strip. The landing is achieved in around 12 seconds, which was deemed reasonable in the context of a real environment, considering the velocity threshold at touchdown and the starting altitude.

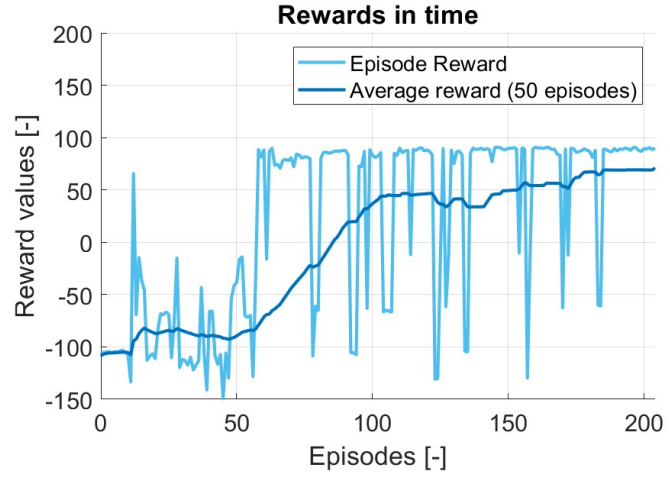


Figure 2.3: Episode rewards and average rewards as function of training episodes for nominal problem setup. Training terminates at 204 episodes when the average reward reaches the value of 70.

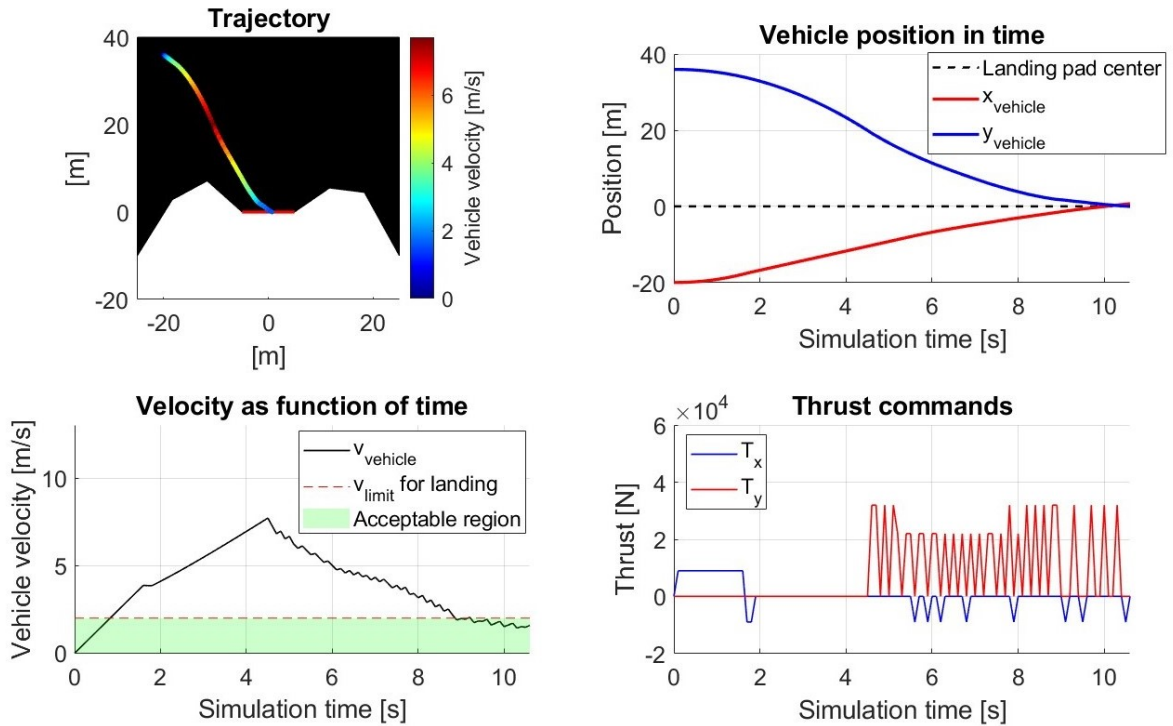


Figure 2.4: Visualization of a successful landing simulation after the Agent has been correctly trained.

2.2. Sensitivity Analysis

Sensitivity analyses were conducted to evaluate the Agent behavior with varying parameter settings, both in the critic settings and in the network structure. By investigating which parameters were given more attention in literature and online examples, changes were applied only to the ones deemed most relevant: α , ϵ , ϵ_{decay} , Mini Batch Size (MBS), neural network layers size, network update frequency. For each of these, two values were chosen on top of the nominal, keeping all other parameters fixed. For every training, results were saved for post-processing and comparisons. The reward functions were not modified for the purpose of this analysis.

It must be noted that this approach falls under the "one-at-a-time" category of sensitivity analyses. These are very useful to assess the individual parameters influences on the problem setup, but fail to highlight any correlation. For instance, changing a single parameter might lead to worse performance, but changing it together with another one might improve on the initial setup. Due to the sheer amount of tunable settings, and the different combinations that could arise, this second type of analysis was not conducted. Fig. 2.5 to 2.10 show the training results of the sensitivity analyses.

As can be seen, a reduction in learning rate still manages to reach convergence, although much slower and with a less smooth behavior. This is likely due to the significantly lower weight given to the temporal difference term in Eq. 1.1. On the other hand, a higher α renders the simulation unstable, impeding convergence. The reason is that, especially at the beginning of the learning process, the Q-function values estimates are far from reality and, if excessive importance is given to them when updating the network, this can prevent correct learning. Similarly, a decrease in ϵ_{decay} yields a successful learning that even improves over the nominal case, while increasing it to 0.05 brings the opposite result. The reason being that $\epsilon_{decay} = 0.5$ is too high of a value, and leads to a premature choice of greedy actions when the Agent has not had enough experiences to properly learn yet. Mini Batch Size appears to require relatively high values, since 512 still manages to converge while 64 does not. Most likely, too small of a MBS does not provide enough information for the gradient calculations during the critic updates. Nonetheless, this is peculiar, since a number of other examples use a MBS of 64, as indicated previously. It must be remembered, however, that differences in parameters values effectiveness is still very dependent on the problem setup both in terms of environment dynamics and rewards definitions, and the setup of this project has indeed a number of differences compared to other online examples.

Moving on, the problem appears extremely sensitive to variations in ϵ , neurons count and network update frequency. For all these settings, a change in either direction prevents the Agent from converging. This was expected for ϵ , given the relevance it has in the Q-function itself (Eq. 1.1), but less for the network parameters which, as mentioned previously, are some of the most varied across different implementations of the Lunar Lander problem. As will be discussed in Chapter 3, deeper investigations would be needed to assess the reasons behind these findings, such as simulations with varying coupled parameters, which would highlight correlations undetected by the "one-at-a-time" approach.

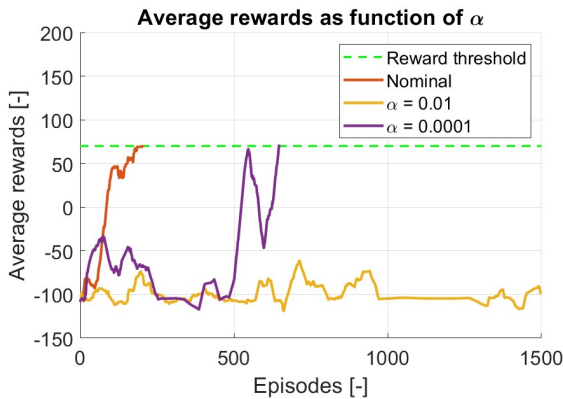


Figure 2.5: Average reward as function of episodes for different α values.

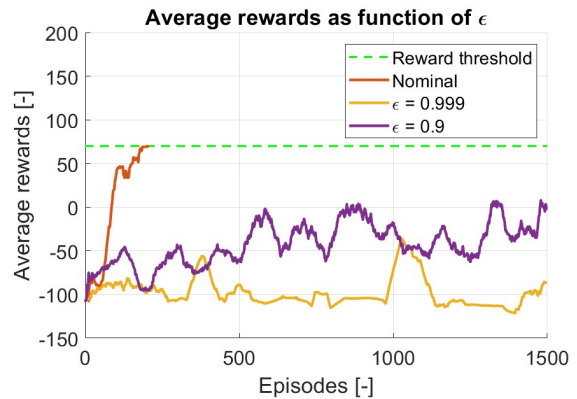


Figure 2.6: Average reward as function of episodes for different ϵ values.

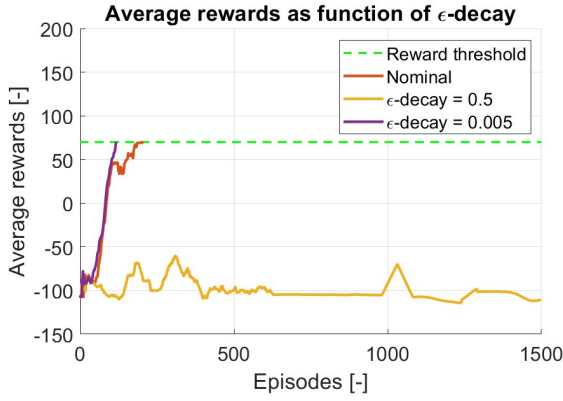


Figure 2.7: Average reward as function of episodes for different ϵ_{decay} values.

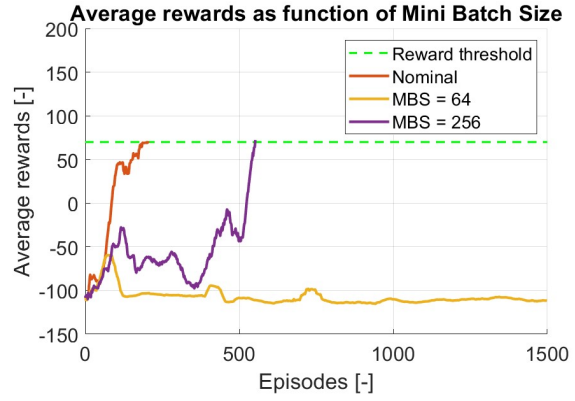


Figure 2.8: Average reward as function of episodes for different Mini Batch Size values.

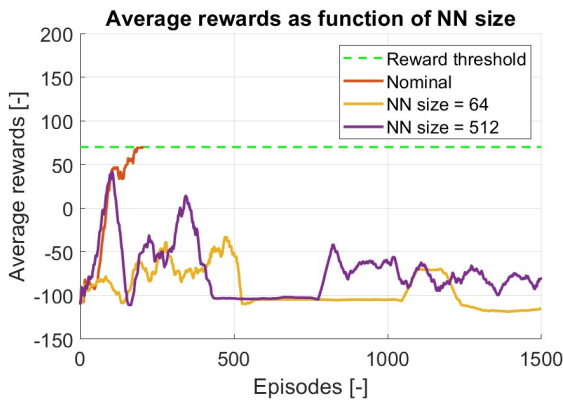


Figure 2.9: Average reward as function of episodes for different network layers sizes.

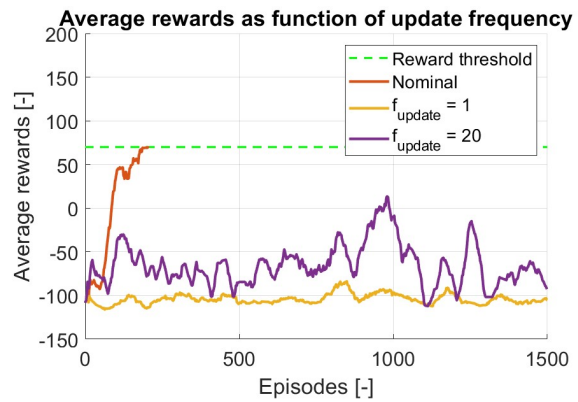


Figure 2.10: Average reward as function of episodes for different network update frequencies.

2.3. Robustness Analysis

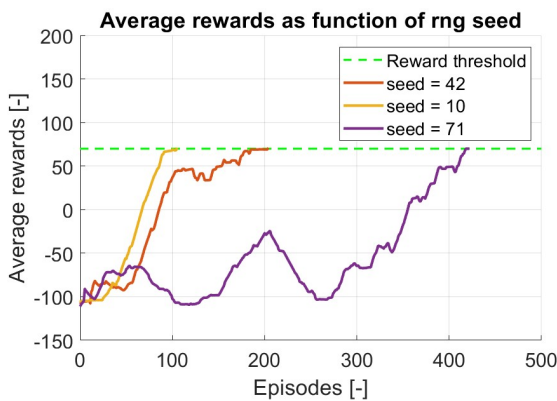


Figure 2.11: Robustness analysis results.

that the values that need to be randomized should, indeed, be random. To verify this, a limited robustness analysis was conducted on the nominal setup by varying the seeds. Fig. 2.11 shows how convergence is reached in all three cases, increasing confidence in the robustness of the algorithm.

Part of the learning process relies on randomized choices, especially in the initialization of the network and in the Agent action choice when the greedy policy is not being employed. In numerical simulations, purely random numbers are impossible to generate, as everything about a code is deterministic in nature. Nonetheless, there are ways to generate pseudo-random numbers, such as using dedicated algorithms. The MATLAB default, and the one which was used for this assignment is the Marsienne Twister. These generators use an integer, also called "seed", to initialize the pseudo-random generation. As such, if the seed is fixed by the user, the generator will always output the same numbers. This was indeed done in the code to ensure reproducibility of results. However, a robust algorithm should work regardless of the seed, because the underlying principle is

Conclusions and Recommendations

The objectives set for this project were to implement a Reinforcement Learning algorithm for a landing simulation on the lunar surface, and to perform several analyses to assess the sensitivity and robustness of the learning process. This was done, firstly, by researching existing implementations of similar problems and studying literature on the underlying principles of RL. Subsequently, the problem was coded using a structured and modular implementation scheme, with the definition of configuration parameters in a different file to ensure the greatest degree of customization. The main script includes the three main blocks of environment definition, critic network setup and Agent parameters tuning. Several functions are called externally to save results and perform post-processing. The first part of the project involved an iterative process which proved necessary to establish the nominal learning setup, given the large number of parameters that can be adjusted and the relative freedom that the RL process allows for. Subsequently, a more systematic approach was employed to perform sensitivity and robustness analyses, with one-at-a-time variations of the desired parameters.

Overall, the initially set goals can be considered met. The environment for the Lunar Lander was correctly defined, and the learning algorithm successfully designed and implemented around it. The quality of the learning results for the nominal case is satisfactory, with a smooth rewards curve and no evident signs of incorrect implementations in any of the settings. Training time and number of episodes also lie in an acceptable range, in line with what had been seen from other examples. The MATLAB Reinforcement Learning Toolbox and the Deep Learning Toolbox helped with the implementation, which would have proven considerably more cumbersome in case the Agent and critic had to be coded from scratch. From the mission point of view, the landing was successfully achieved when simulating the trained agent, with all constraint conditions met. Trajectory parameters appeared reasonable and in line with expectations. This implies that the Agent and its network setup were designed correctly, managing to learn the appropriate control law needed to land a vehicle in a 2D airless environment.

The extent of this project, in terms of mission fidelity, was clearly simplistic. Three main improvements should be implemented if the end goal is to start applying learning algorithms for spacecraft GNC in real-life operations. From the environment point of view, upgrading to 3D rotational dynamics would considerably increase the realism of the simulations, although carrying a growth in the computational complexity as well. The equations of motion would rise from two to six, carrying the need for a deeper and wider network, in turn increasing the time needed to simulate each episode. Fuel should also be taken into consideration. A mass-varying system requires mass as an additional state element but, even more importantly, limits the exploration freedom of the Agent. This is because burning all the available fuel would end an episode in the same way that crashing on the ground does. As such, the Agent would have an additional constraint when performing the landing, possibly rendering invalid some trajectories that work in the infinite-fuel case. Finally, a finer level of throttle control is necessary to simulate spacecrafts' propulsion systems more realistically, both the main engine and the side thrusters. This implies the need of a continuous action space, which cannot be handled by DQN. Actor-Critic (AC) or Proximal Policy Optimization (PPO) are two commonly used learning approaches for continuous-space and continuous-actions problems.

As for the results analyses, combined tuning would undoubtedly give more insights on parameters correlations, shading light on the causes behind some behaviors that are still unexplained after the "one-at-a-time" approach.

References

- [1] Neuromatch Academy. *Performance Analysis of DQN Algorithm on the Lunar Lander task*. URL: https://deeplearning.neuromatch.io/projects/ReinforcementLearning/lunar_lander.html (visited on 12/24/2020).
- [2] L. Zhou et al. "The Expressive Power of Neural Networks: A View from the Width". In: *31st Conference on Neural Information Processing Systems* (2017). Long Beach, CA, USA.
- [3] N. Dhoble et al. *Lunar Lander - Deep Reinforcement Learning, Noise Robustness, and Quantization*. 2000. URL: <https://xusophia.github.io/DataSciFinalProj/>.
- [4] andrejlukic. *lunar_lander_ddqn*. 2023. URL: <https://github.com/andrejlukic/lunar-lander-ddqn>.
- [5] Richard S. Barto and Andrew G. Barto. *Reinforcement Learning: An Introduction*. 2th ed. The MIT press Ltd., 2018.
- [6] V. Bons. *Solving Lunar Lander using DQN with Keras*. 2020. URL: <https://wingedsheep.com/lunar-lander-dqn/>.
- [7] chenjnelson. *DDQN Lunar Lander*. 2019. URL: https://github.com/chenjnelson/DDQN_LunarLander.
- [8] Cornell University. *Adam*. 2021. URL: <https://optimization.cbe.cornell.edu/index.php?title=Adam>.
- [9] Katnoria. *DQN Implementation: Solving Lunar Lander*. 2000. URL: https://www.katnoria.com/nb_dqn_lunar/.
- [10] S. Levine. *Decisions from Data: How Offline Reinforcement Learning Will Change How We Use Machine Learning*. 2020. URL: <https://medium.com/@sergey.levine/decisions-from-data-how-offline-reinforcement-learning-will-change-how-we-use-ml-24d98cb069b0>.
- [11] MathWorks. *Deep Learning Toolbox*. URL: <https://nl.mathworks.com/products/deep-learning.html>.
- [12] MathWorks. *Reinforcement Learning Agents*. URL: <https://nl.mathworks.com/help/reinforcement-learning/ug/create-agents-for-reinforcement-learning.html>.
- [13] MathWorks. *Reinforcement Learning Toolbox*. URL: <https://nl.mathworks.com/products/reinforcement-learning.html>.
- [14] MathWorks. *rldqnAgentOptions*. 2019. URL: <https://nl.mathworks.com/help/reinforcement-learning/ref/rldqnagentoptions.html>.
- [15] Aditi Adesh Shenoy. "Feature optimization of contact map predictions based on inter-residue distances and U-Net++architecture". In: *Stockholm University* (2019).
- [16] L. Wang. *Training the Lunar Lander Agent With Deep Q-Learning (DQN) and Double DQN*. 2021. URL: <https://medium.datadriveninvestor.com/training-the-lunar-lander-agent-with-deep-q-learning-and-its-variants-2f7ba63e822c>.